

SWIM: *S*可扩展的*W*完全一致的*I*感染式流程组*M*成员协议

Abhinandan Das、Indranil Gupta、Ashish Motiwal^{*} 康奈尔大学计算机科学系 美国伊萨卡 NY 14853 {asdas、gupta、ashish}@cs.cornell.edu

抽象的

Several distributed peer-to-peer applications require weakly-consistent knowledge of process group membership information at all participating processes. SWIM is a generic software module that offers this service for large-scale process groups. The SWIM effort is motivated by the unscalability of traditional heart-beating protocols, which either impose network loads that grow quadratically with group size, or compromise response times or false positive frequency w.r.t. detecting process crashes. This paper reports on the design, implementation and performance of the SWIM sub-system on a large cluster of commodity PCs.

Unlike traditional heartbeating protocols, SWIM separates the failure detection and membership update dissemination functionalities of the membership protocol. Processes are monitored through an efficient peer-to-peer periodic randomized probing protocol. Both the expected time to first detection of each process failure, and the expected message load per member, do not vary with group size. Information about membership changes, such as process joins, drop-outs and failures, is propagated via piggybacking on ping messages and acknowledgments. This results in a robust and fast infection style (also epidemic or gossip-style) of dissemination.

The rate of false failure detections in the SWIM system is reduced by modifying the protocol to allow group members to suspect a process before declaring it as failed - this allows the system to discover and rectify false failure detections. Finally, the protocol guarantees a deterministic time bound to detect failures.

Experimental results from the SWIM prototype are presented. We discuss the extensibility of the design to a WAN-wide scale.

^{*} Author last names are in alphabetical order. The authors were supported in part by NSF CISE grant 9703470, in part by DARPA/AFRL-IFGA grant F30602-99-1-0532, and in part by a grant under NASA's REE program, administered by JPL.

一、简介

*As you swim lazily through the milieu,
The secrets of the world will infect you.*

多个在互联网上运行的大规模点对点分布式进程组依赖于分布式成员维护子系统。利用成员协议的现有中间件系统的例子包括可靠的多播[3,11]和流行式信息传播[4,8,13]。这些协议反过来又可用于需要协调最近断开连接的更新的分布式数据库[14]、发布-订阅系统和大规模点对点系统[15]等应用程序。其他新兴应用程序（例如大规模合作游戏和其他协作分布式应用程序）的性能主要取决于其中使用的成员维护协议的可靠性和可扩展性。

简而言之，成员协议为组中的每个进程（“成员”）提供本地维护的组中其他无故障进程的列表。该协议确保成员列表随着新成员加入组或退出（自愿或因失败）而发生的变化进行更新。应用程序可以直接在其地址空间中使用成员列表，也可以通过回调接口或API来使用成员列表。应用程序可以根据需要免费使用列表的内容，例如基于八卦的传播协议将使用该列表定期挑选八卦的目标成员。

会员子系统的可靠性和可扩展性可以通过几个性能指标来衡量。成员变更发生后必须在组内迅速传播。底层网络的异步性和不可靠性可能会导致消息丢失，从而导致进程故障的错误检测，因为丢失消息的进程与失败的进程无法区分[10]。这种误报率必须很低。最后，该协议需要是点对点的（不依赖于中央服务器），并且对网络和进程施加较低的消息和计算负载。

成员协议很难在超过几十个进程的组中扩展[11, 16], 从而影响使用它们的应用程序的性能。正如[16]中所报告的, 这些组规模下性能不佳的主要症状是流程错误故障检测率或检测故障时间的增加。[12]将此类成员协议所施加的消息负载的二次增加视为传统成员维护协议不可扩展的另一个症状。严重依赖成员子系统的应用程序的一个示例是虚拟同步多播协议类 [3]。当超过几十个成员时, 该规范的传统实现会导致性能和分区急剧下降[11]。

本文介绍了我们在 SWIM 项目中实现成员子系统的努力, 该系统提供稳定的故障检测时间、稳定的误报率和每个组成员的低消息负载, 从而允许使用它的分布式应用程序能够很好地扩展。我们关注组成员资格的较弱变体, 其中不同成员的成员资格列表不需要在同一(因果)时间点在整个组中保持一致。可以通过扩大会员子系统来提供更有力的保证, 例如可以通过定期检查成员资格列表的定序器进程来提供虚拟同步类型的成员资格。然而, 与弱一致问题不同, 强一致规范可能具有基本的可扩展性限制¹。

分布式成员资格算法的设计传统上是通过 *heartbeating* 技术来实现的。每个进程定期向外界发送一个递增的心跳计数器。当一段时间内未收到另一个进程的心跳时, 该进程将被检测为失败。然而, 心跳的实际实现受到可扩展性限制。将所有心跳发送到中央服务器会导致热点创建。向所有成员发送心跳(通过网络多播或八卦 [16])会导致网络和组上的消息负载随着组大小呈二次方增长。当存在多个故障时, 沿着逻辑环[9]的心跳会受到故障检测时间的不可预测性的影响。不幸的是, 随着组规模的增加, 同时发生多个故障的可能性也随之增加。

关于基于心跳的成员维护机制固有的不可扩展性背后的原因的扩展讨论可以在[12]中找到。这篇论文还提出了一种基于成员随机探测对方而不是心跳的随机分布式故障检测器协议——

Discussion of this issue is outside the scope of this paper. The reader is referred to [11].

A “weakly-consistent” adjective is implicitly assumed, and dropped henceforth.

ing。数学分析表明, 随着组规模的扩大, 协议的(预期)故障检测时间、误报率和每个成员的消息负载等属性都与组规模无关。这是对基于全部心跳的协议的改进, 该协议的故障检测时间或每个成员的网络带宽使用量(或误报率增加)呈线性变化(随组大小变化)。

我们在本文中的工作是受[12]工作的启发, 即流行的全对全心跳协议类的不可扩展性源于其中隐式决定融合成员问题规范的两个主要功能: 1) 成员资格更新 *Dissemination*: 传播因进程加入、离开或失败而产生的成员资格更新, 以及2) *Failure detection*: 检测现有成员的失败。通过设计一个高效的基于非多播的故障检测器, 并且仅在成员资格发生更改时才使用传播组件, 可以消除多播心跳的开销。成员资格传播组件可以通过硬件多播或感染方式来实现。

虽然[12]提出了一种故障检测协议并对其进行了理论上的分析, 但我们在当前论文中的工作着眼于将成员资格传播组件纳入其中以构建一个工作成员资格系统。此外, 所得协议还通过降低误报率的机制得到增强, 并对各个进程的故障检测时间提供更强的确定性保证。

我们的系统称为 SWIM, 提供了一个会员基础:

- (1) 对每个组成员施加恒定的消息负载; (2) 在(预期)恒定时间内检测到组中 *some* 个无故障进程的进程故障;
- (3) 提供一个非故障进程检测另一个进程故障所需的本地时间的确定性界限(作为组大小的函数);
- (4) 传播成员更新, 包括有关失败的信息, 也采用八卦风格或流行病风格 [2, 8]); 组内的传播延迟随着成员数量的增加而缓慢(对数)增长;
- (5) 提供了一种机制, 通过在组内“宣布”某个进程失败之前“怀疑”该进程来降低误报率。

虽然 (1) 和 (2) 是[12]的故障检测协议的属性, (3) - (5) 代表了我们在当前论文中的后续工作。讨论了在 PC 集群上运行的 SWIM 原型实现的实验结果。SWIM 协议也可以扩展工作

In a sense, the protocol monitors the status of members, randomly, instead of using heartbeating.

通过广域网 (WAN) 或虚拟专用网 (VPN), 我们将在第 6 节中简要介绍这一点。

本文的其余部分组织如下。第 2 节总结了该领域之前的工作, 以及 [12] 中可扩展故障检测协议的基础知识。第 3 节描述了基本的 SWIM 协议, 第 4 节描述了协议的改进。第 5 节介绍了原型实现的实验结果。我们在第 6 节中得出结论。

2. 之前的工作

在传统的分布式全对所有心跳故障检测算法中, 每个组成员定期向所有其他组成员发送“心跳”消息 (带有递增的计数器)。当 j 在连续的心跳周期内没有收到来自 i 的心跳时, 成员 i 被无故障成员 j 声明为故障。

分布式心跳方案保证在任何非故障成员上始终检测到故障成员 (在故障后的时间间隔内), 因为崩溃的成员也会停止发送心跳消息。然而, 这些协议的准确性和可扩展性保证有所不同, 具体取决于用于传播心跳的实际机制。

在最简单的实现中, 每个心跳都被多播到所有其他组成员。这会导致每秒 $(\frac{2}{t_{ssip}})$ 条消息的网络负载 (即使使用 IP 多播), 其中 T 是分布式应用程序所需的故障检测时间。van Renesse 等人 [16] 提出通过稳健的八卦式协议传播心跳。在此协议中, 每个 t_{gssip} 时间单位, 每个成员都会向一些随机目标进行闲聊, 即从其他成员收到的最新已知心跳计数器的 (n) 大小的列表。虽然闲聊减少了误报频率, 但按照预期, 新的心跳计数通常需要 $[l(n) t_{gssip}]$ 时间单位才能到达任意其他组成员。为了满足应用程序指定的检测时间, 协议每秒生成 $(\frac{2}{t_{ssip}} l g(n))$ 字节的网络负载。使用消息批处理来解决此问题受到 UDP 数据包大小限制的限制, 例如 50 个成员的 5B 心跳 (IP 地址和计数) 已经占用 250 B, 而 SWIM 生成的数据包大小最多为 135 B, 无论组大小如何。

网络负载的二次增加是由于向 *all* 组成员发送心跳通知而导致的。通过将失败的 *de-tection* 操作与成员资格更新 *dissemi- nation* 操作分开可以避免这种情况。

已经提出了几种等级会员制度, 例如国会 [1]。这属于更广泛的一类解决方案, 其中每个进程仅对一个进程子组进行检测。此类协议需要仔细配置和维护会员信息流动的覆盖层, 并且协议的准确性取决于该图的稳健性。相比之下, SWIM 的设计避免了虚拟图的开销。

SWIM 对上述不可扩展性问题的解决方案基于 (a) 分别设计故障检测和成员更新传播组件, 以及 (b) 使用基于非心跳的策略进行故障检测。

在继续描述 SWIM 协议内部结构之前, 我们首先为理解分布式故障检测器协议的效率和可扩展性的关键特征奠定基础。几项研究 [6,7,12,16] 已经确定了分布式故障检测器协议的这些基本属性 (从理论和实践角度), 以及与同时满足它们相关的不可能结果。由此产生的权衡通常由分布式应用程序所需的安全性和活性决定。

这些属性是 [12]:

(1) *Strong Completeness*: 任何组成员的崩溃故障被所有非故障成员检测到 [6]; (2)

Speed of failure detection: 成员故障与 *some* 非故障组成员检测到之间的时间间隔; (3) *Accuracy*: 故障检测误报率; (4) *Network Message Load*, 由协议生成的每秒字节数。

[6] 证明了不可能通过异步网络构建既准确 (无错误检测) 又高度完整的故障检测器。然而, 由于典型的分布式应用程序依赖于始终保持的强完整性 (以便在动态组中维护最新信息), 因此大多数故障检测器 (包括基于心跳的解决方案) 都保证了此属性, 同时尝试保持较低的误报率。SWIM 采用同样的方法。

在 [12] 中, 一个简单的计算确定了满足每个成员的错误检测率的指定参数所需的最小总网络负载 (每秒字节数) (表示为

$(\frac{2}{t_{ssip}} l g(n))$ 和大小为 n 的组中的检测时间 (T)。[12] 将此负载计算为 $n \frac{l g(n)}{l g(p_{ml})}$, 其中 p_{ml} 是底层网络内丢包的概率。

尽管此计算是在每条消息 (p_{ml}) 上独立消息丢失概率的理想条件下完成的, 但它可以作为比较不同故障检测协议的可扩展性的良好基线。例如, 第 2 节中讨论的全对全心跳协议有一个次优因子, 该因子随

⁴This property is called *Strong Completeness*.

group size.

3. 基本的 SWIM 方法

如前所述, SWIM 方法有两个组成部分:

(1) *Failure Detector Component*, 检测成员的失败; (2) *Dissemination Component*, 传播有关最近加入或离开群组或失败的成员的信息。

现在我们通过描述基本的 SWIM 协议来奠定基础。基本协议使用[12] (第3.1节) 基于随机探测的故障检测器协议, 并通过网络多播传播成员资格更新 (第3.2节)。SWIM 协议是在后续部分 (第4节) 中通过改进初始设计而开发的。

3.1. SWIM故障检测器

SWIM 故障检测器算法[12]使用两个参数: 以时间单位表示的协议周期 T (和整数 k , 即故障检测子组的大小)。该协议不需要在成员之间同步时钟, 并且如果 T 是组成员的平均协议周期, 则协议的属性保持不变。

图1说明了协议在任意成员 M_i 上的工作情况。在长度为 T 时间单位 (在 M_i 的本地时钟上) 的每个协议周期内, 从 M_i 的成员资格列表 (例如 M_j) 中随机选择一个成员, 并向其发送一条 ping 消息。 M_i 然后等待来自的回复确认

M_j 。如果在预先指定的超时 (由消息往返时间决定, 该时间选择小于协议周期) 内未收到此消息, 则 M_i 间接探测 M_j 。 M_i 随机选择 k 成员并向每个成员发送一条 ping-req(M_j) 消息。这些成员中的每一个 (无故障的成员) 在收到此消息后, 都会对 M_j 执行 ping 操作, 并将来自 M_j (if returned) 的确认转发回

M_i 。在此协议周期结束时, M_i 检查是否已直接从 M_j 或间接通过 k 成员之一收到任何确认; 如果不是, 它会在其本地成员列表中声明 M_j 失败, 并将此更新传递给传播组件。

在图1的示例中, 当 M_j 启动时, k 成员之一设法完成此事件周期, 并且在该协议周期结束时, M_i 并不怀疑 M_j 有故障。

用于启动间接探测的预先指定的超时基于对网络内往返时间分布的估计, 例如可以使用平均值或99th百分位。请注意, 协议周期 T 必须至少是往返估计的三倍。在我们的实验中, 我们使用平均测量的往返时间来设置

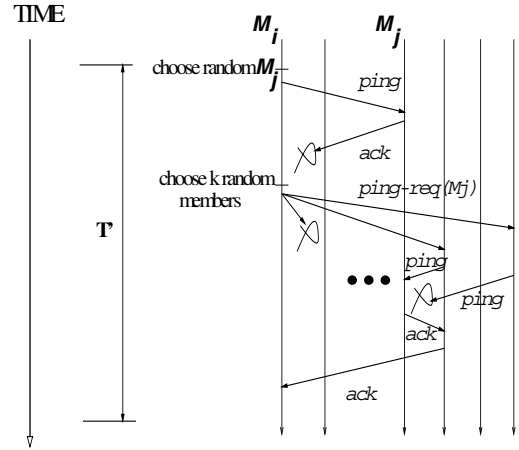


图1. SWIM 故障检测: M_i 的协议周期示例。这显示了协议周期可能发起的所有可能的消息。为简单起见, 排除了一些消息内容。

超时, 而我们的协议周期明显大于该值。

该协议的每条消息中包含的数据都在发起者 (M_i) 处标有协议周期的唯一序列号。请注意, ping、ping-req、ack 消息的大小受常数限制, 并且与组大小无关。

上述协议的第二部分使用成员的间接探测子组来中继 ping 和 ack。使用这种方法的基本原理是, 避免 M_i 和 M_j 之间的网络路径上的任何拥塞影响, 而不是直接向 M_j 发送 k ping 消息, 或者直接将 ping-reqs 的 ack 转发回 M_i ; 这可能导致原始 ping 消息或其 ack 被丢弃。

该故障检测器协议在[12]中进行了分析。在这里, 我们总结一下分析结果:

如果每个成员都有一个大小为 n 的成员资格列表, 并且其中一部分是无故障的, 则在协议周期内任意成员被选为 ping 目标的可能性为 $1 - (1 - \frac{1}{n})^{-1}$, 该可能性快速下降 (并且渐近为 $\frac{1}{n}$) 到 $1 - e^{-\frac{1}{n}}$ 。

因此, 组中任意成员发生故障与被 some 进程检测到之间的预期时间最多为 $T \cdot \frac{1}{1 - e^{-\frac{1}{n}}}$ 。这根据应用程序指定的预期检测时间给出了协议周期长度的估计。

如果 m_l 是网络及时传送数据包的概率, 独立于所有数据包, 则任意非故障成员将被错误地检测为故障

⁵The reader is encouraged to work out these results, or refer to [12].

概率为 $(1 - m_l)(1 - m_l)$ 的协议周期

根据应用程序所需的误报概率给出了 k 的可配置值。

该故障检测器满足强完整性：故障成员最终将被选择为每个非故障成员的 ping 目标，并从其成员列表中删除。

协议对每个成员施加的预期消息负载是一个常数，不随组大小而变化，并且在所有成员之间是对称的。该负载可以根据 k 的估计来计算。

这些属性都不依赖于（除了渐近的）组大小 n 。

3.2. 传播组件和动态成员资格

在检测到另一个组成员的故障时，该过程只需将此信息作为 *failed*(j) 消息多播到组的其余成员。收到此消息的成员将从其本地成员列表中删除 j 。

关于新加入的成员或自愿离开的成员的信息以类似的方式多播。然而，对于加入该组的进程来说，它需要知道该组中至少一个联系人成员。这可以通过以下几种方式之一来实现：如果该组与众所周知的服务器或 IP 多播地址相关联，则所有加入都可以定向到相关联的地址。在缺乏此类基础设施的情况下，可以广播加入消息，并且听到该消息的组成员可以概率性地决定（通过抛硬币）是否回复该消息。或者，为了避免多个成员回复，可以在组内维护静态协调器以处理组加入请求。事实上，多个协调器的存在并不影响协议的正确性，只会导致对加入请求的多个回复。随着时间的推移，可以通过传播组件来发现和解决多个协调员。在当前版本的 SWIM 中，我们选择保留一个协调员，尽管没有理由排除任何其他策略。

4. 更稳健、更高效的 SWIM

第 3 节描述了使用网络多播传播成员资格更新（由于成员加入、离开或失败而产生）的基本 SWIM 协议。然而，网络多播原语（例如 IP 多播等）只是尽力而为 - 网络内的消息丢失可能会导致任意且相关的成员资格未收到

任何组成员的变化。在第 4.1 节中，我们描述了传播组件的设计，该组件在故障检测器协议发送的 ping 和 ack 消息上搭载成员资格更新。这完全消除了传播组件（即多播）生成额外数据包的情况。SWIM 生成的唯一数据包是 ping、ping-reqs 和 ack，从而为每个组成员提供恒定的预期消息开销。这种方法会产生感染式的传播，并具有对数据包丢失的鲁棒性和低延迟的相关优势。

基本的 SWIM 故障检测器协议尽管具有可计算的准确性，但仍会受到缓慢进程的影响（例如，由于缓冲区溢出而丢失大量数据包），从而将其他几个无故障的进程声明为有故障。进程也可能在短时间内受到干扰，例如在超载的主机上。这可能会导致进程错过及时回复同时收到的 ping 的机会，并被错误地声明为失败。第 4.2 节介绍了 *Suspicion* 机制，其中对第 3 节中描述的 SWIM 故障检测器协议生成的 ping 消息无响应的进程不会立即声明为“故障”。相反，该进程被声明为“可疑”，并且该信息使用传播组件在整个组中传播。在预先指定的超时后（我们在第 5 节中讨论该参数的值），可疑进程被声明为“故障”，并将此信息传播给小组。但是，如果可疑进程在此超时到期之前响应 ping 请求，则有关此信息的信息将作为“活动”消息传播到组。然后，该过程会在不同成员的成员列表中重新焕发活力，而无需离开或重新加入该组。因此，这种预先指定的超时有效地权衡了故障检测时间的增加和错误故障检测频率的减少。

基本的 SWIM 故障检测协议保证在每个非故障组成员 j 处最终检测到任意进程 i 的故障。然而，它对任意成员 i 失败和在另一个任意成员 j 上检测到它之间的时间（以 j 上的本地协议轮数而言）没有提供确定性保证。4.3 节描述了对原始 SWIM 故障检测器协议的修改，以保证这样的

Time Bounded Completeness 属性；故障发生与成员 j 检测到故障之间的时间间隔不超过组大小的两倍（以协议周期数计）。

4.1. 感染型传播成分

第 3 节的基本 SWIM 协议使用多播原语在组中传播成员资格更新。硬件组播和 IP 组播可用

Absence of centralization is a common design philosophy in peer-to-peer systems today.

大多数网络和操作系统，但很少启用，例如出于管理原因。然后，基本的 SWIM 协议将不得不使用昂贵的广播或低效的点对点消息传递方案，以便将成员资格更新传播给所有小组成员。此外，由于这种多播是不可靠的，因此只能尽最大努力将成员资格更改传播到该组。

相反，增强的 SWIM 协议完全消除了外部组播原语的使用。它通过搭载要在故障检测器协议生成的 ping、ping-req 和 ack 消息上传播的信息来实现这一点。我们将其称为感染式传播机制，因为信息的传播方式类似于社会中的八卦传播或普通人群中的流行病[8]。请注意，传播组件的此实现不会生成任何额外的数据包（例如多播）- 传递给该组件的所有“消息”均通过搭载故障检测组件的数据包来传播。

Bailey [2] 对流行病在 n 成员的均匀混合组（其中有一个初始感染成员）内的传播进行了确定性分析。在每个时间单位的接触率为 1 的情况下，（预期）感染成员数量 x （初始为 1）与时间 t 之间的关系为：

$$\frac{dx}{dt} = x(n-x) \quad x = \frac{n}{1 + (n-1)e^{-t}}$$

在我们的感染式传播组件中，可以以类似的方式分析通过 ping 和 ack 消息传播成员资格更新。以协议周期为时间单位，接触率是任意一对感染和未感染成员之间接触的概率，等于 $[1 - (1 - \frac{1}{n})^n] = (1 - \frac{1}{n})^n$ 。这给了我们 $x = \frac{n}{1 + (n-1)(1 - \frac{1}{n})^n}$ 。

这种流行病过程在群体中呈指数级快速传播；在协议的 $t = \lambda l_n$ 轮之后（其中 λ 是参数），受感染成员的预期数量为 $x = \frac{n}{1 + (n-1)(1 - \frac{1}{n})^n} \geq n(1 - \frac{1}{(2-1/n)^n})$ 。因此，通过捎带以感染方式传播的成员更新将在 λl_n 协议周期后到达 $(n - n^{-(1-1/n)})$ 组成员。简单来说，随着 n 的增加（以及 λ ）， x 的估计值将变为 $(n - n^{-(1-1/n)})$ 。将 λ 设置为一个小的常数足以可靠地传播流行病 - 即使在小群体规模下也是如此，正如我们在第 5 节中的实验所证明的那样。

该文献包含对其他几种流行病类型的分析[4,8,13]，关于其概率可靠性的结论基本相似。这些分析还表明，传播的感染方式对于网络内的流程故障和消息丢失具有弹性，

就像流行病的传染性一样。我们实施的实验结果展示了这些特征。

关于实施的一句话是有必要的。每个组成员 i 处的 SWIM 协议层维护最近成员资格更新的缓冲区，以及每个缓冲区元素的本地计数。本地计数指定到目前为止该元素已被 i 搭载的次数，并用于选择接下来要搭载的元素。每个元素最多被搭载 λl_n 次。如果此缓冲区的大小大于单个 ping 消息（或 ack）上可以搭载的最大元素数，则优先选择已传播次数较少的元素。这是必要的，因为协议周期是固定的，并且成员变化的速度可能会暂时压倒传播的速度。在这种情况下，首选“年轻”的缓冲元素可以确保所有成员变更至少影响一些成员——当成员变更注入率停止时，这些变化将传播到组的其他成员。

我们对该协议的实现维护了两个组成员列表——一个是组中尚未声明为失败的成员列表，另一个是最近失败的成员列表。目前，从这两个列表中选择相同数量的元素缓冲区进行搭载，但该方案可以推广以适应流程加入、离开和失败率的相对变化。

4.2. 怀疑机制：减少误报频率

在到目前为止描述的 SWIM 故障检测器协议中，如果一个无故障的组成员 j 被另一个组成员 i （错误地）检测为失败，无论是由于网络数据包丢失还是因为 j 休眠了一段时间，或者因为 i 是一个缓慢的进程，那么 j 将在组中被声明为失败。换句话说，一个完全健康的进程 j 会遭受非常严重的惩罚，因为一旦被错误地检测为组中失败，就被迫退出组。这导致检测故障时误报率很高。

我们通过修改 SWIM 来减少这个问题的影响，只要基本 SWIM 故障检测器协议检测到故障，就运行一个称为 *Suspicion* 子协议的子协议。

Suspicion 子协议的工作原理如下。考虑成员 i 选择成员 j 作为当前协议周期内的 ping 目标，并运行基本 SWIM 故障检测器协议周期。如果 i 没有直接或通过间接探测子组收到确认，则它不会声明 j 失败。相反， i 将 j 标记为 i 本地成员资格列表中的 *Suspected* 成员。此外， $\{Suspect_j: i \text{ suspects } j\}$ 消息通过传播组件在整个组中传播（在我们系统中以感染方式）。任何

收到此类消息的组成员 i 也会将 j 标记为可疑。可疑成员保留在成员列表中, 并在 SWIM 故障检测器协议的 ping 目标选择操作方面受到与无故障成员类似的对待。

如果成员 i 在基本 SWIM 协议的适当过程中成功 ping 到可疑成员成员 j , 它就会在其成员列表中取消标记先前对 j 的怀疑, 并通过传播组件在组中传播 $\{Alive\ j: i\ knows\ j\ is\ alive\}$ 消息 (在我们系统中以感染方式)。这样的 Alive 消息在接收成员的成员列表中取消标记可疑成员 j 。请注意, 如果成员 j 收到这样一条怀疑的消息, 它可以开始传播一条 Alive 消息, 以澄清其未失败。

成员资格列表中的可疑条目将在预先指定的超时后过期。如果某个成员 i 怀疑 j , 并且此条目在收到 Alive 消息之前超时,

声明 j 有故障, 将其从本地成员列表中删除, 并开始通过传播组件传播消息 $\{Confirm\ j: i\ declares\ j\ as\ faulty\}$ 。此消息将覆盖任何先前的 *Suspect* 或 *Alive* 消息, 并从所有收件人的成员资格列表中级联删除 j 。

这种机制降低了 (但没有消除) 故障检测误报率。还要注意, 原始协议的强完整性属性继续保持。怀疑失败的进程 j 的进程的失败可能会延长检测时间, 但最终的检测是有保证的。

根据上面的讨论, *Alive* 消息覆盖 *Suspect* 消息, 而 *Confirm* 消息覆盖 *Suspect* 和 *Alive* 消息, 它们对与可疑成员对应的本地成员资格列表元素有影响

j 。然而, 一个成员在其一生中可能会多次被怀疑和未被怀疑。这些多个版本的 *Suspect* 和 *Alive* 消息 (都属于同一成员 j) 需要通过唯一标识符来区分。这些标识符是通过对成员列表中的每个元素使用虚拟 *incarnation number* 字段来提供的。转世数字是全球性的。会员

i 当它加入组时, 它的化身号被初始化为 0, 并且当它接收到 (通过传播组件) 有关其自身在当前化身中被怀疑的信息时, 它只能增加 $i - i$ 然后生成一条带有其标识符和递增的化身号的 *Alive* 消息, 并通过传播组件将其传播到组。

因此, *Suspect*, *Alive*, 和 *Confirm* 消息除了其标识符之外还包含成员的化身编号。下面详细说明了这些消息的优先顺序及其对成员列表的影响。

- $\{Alive\ M_i, inc = i\}$ overrides

- $\{Suspect\ M_i, inc = j\}, i > j$
- $\{Alive\ M_i, inc = j\}, i > j$

- $\{Suspect\ M_i, inc = i\}$ 覆盖

- $\{Suspect\ M_i, inc = j\}, i > j$
- $\{Alive\ M_i, inc = j\}, i \geq j$

- $\{Confirm\ M_i, inc = i\}$ 覆盖

- $\{Alive\ M_i, inc = j\}, any\ j$
- $\{Suspect\ M_i, inc = j\}, any\ j$

很容易看出, 这些优先顺序和优先顺序保持了故障检测器组件所需的正确性属性。熟悉 AODV [5] 等自组织路由协议的读者会注意到它们对目的地序列号的使用与我们的化身号方案之间的相似性。

偏好规则和感染式传播组件还适应多个其他进程对一个进程的怀疑。偏好规则不依赖于怀疑源, 如果存在多个源, 则感染式传播会更快地传播消息 (*Suspect*, *Alive* 或 *Confirm*), 每个进程的开销与单个污染源完全相同 [8]。

4.3. 循环探测目标选择: 提供有时间限制的强完整性

第 3 节中描述的基本 SWIM 故障检测器协议在平均恒定数量的协议周期中检测故障。尽管每个进程故障都保证最终会在所有其他无故障进程中被检测到 (最终的强完整性), 但对整个组中 ping 目标的病态选择可能会导致在组中任何地方首次检测到进程故障时出现很大的延迟。在极端情况下, 这种延迟可能是无限的, 因为失败的进程可能永远不会被任何其他非故障进程选为 ping 目标。

这可以通过对协议进行以下修改来解决。成员 i 的故障检测协议通过维护当前成员列表的已知元素的列表 (直观上是一个数组) 来工作, 并且不是从该列表中随机而是以循环方式选择 ping 目标。相反, 新加入的成员被插入到成员列表中统一随机选择的位置。完成对整个列表的遍历后, i 会将成员资格列表重新排列为随机重新排序。

考虑在成员 i 处执行如上所述修改的 SWIM 协议。一旦另一个成员 j 包含在 i 的成员资格列表中, 则在每次遍历 i 的成员资格列表期间, 它将被选为 ping 目标一次。如果成员列表的大小不超过 n_i , 则对同一目标的连续选择最多相隔 $(2n_i - 1)$ 个协议周期。这限制了

任何成员的进程故障的最坏情况检测时间为 t_i ，从而满足 *Time Bounded* 完整性属性。

此优化保留了原始协议的平均故障检测时间，因为组中不同成员的成员列表的随机化导致每个成员的 ping 目标选择的分布相似。

5. 原型性能评估

SWIM 协议的原型是通过 Winsock 2 API 实现的，并在运行 Windows 2000 的大型商用 PC 集群中进行了测试。该 PC 集群由 16 450 MHz Dell PII、16 个 1 GHz IBM x220 以及一组双节点和四节点（200 MHz 至 500 MHz PII 和 PIII 处理器）组成，通过 100 Mbps 以太网进行通信，无外部负载。每个节点最多包含一个进程组成员。

实验参数设置如下。为 ping-reqs 选择的成员数量为 $K = 1$ ，使用的协议周期为 2 秒。每次感染（成员更新）最多都会搭载每个成员发送的最多 $(3 l (N + 1))$ 条消息。通过这个值设置，我们的实验运行没有观察到成员或非自愿分区的永久部分成员资格列表的证据。将可疑成员声明为失败的怀疑超时也设置为该值。

我们比较了该协议的三个不同版本：（1）（*SWIM:Basic*）第3节的基本SWIM协议，用第4.3节中描述的循环方案进行修改，（2）（*SWIM+Inf.*）带有感染式传播组件的SWIM（如第4.1节中所述），以及（3）（*SWIM+Inf.+Susp.*）*SWIM+Inf.*。与第 4.2 节中描述的怀疑子协议扩展。

SWIM协议发送的所有点对点消息都是UDP数据包。SWIM:Basic 中的最大消息负载大小为 15 B，SWIM+Inf 中的最大消息负载大小为 135 B。和游泳+信息。+暂停。协议（因为每条消息最多搭载 6 个成员资格更新）。

5.1. 消息负载

图 2 显示了 SWIM 故障检测器在任意组成员处测量的消息发送和接收负载。读数是在跨越 40 个协议周期的时间段内获取的。团体规模不超过 55 名成员时，平均开销保持在 2.0 左右。这与分析估计相匹配，因为在每个协议周期内，组成员发送单个 ping 并接收

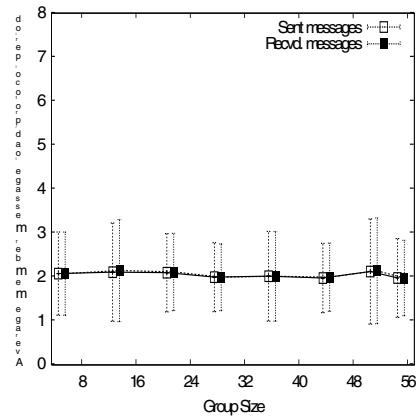


图 2. 组成员发送和接收消息的开销。为了清晰起见，点水平扰动。

相应的 ack，以及接收一条 ping 消息（按预期）并发送相应的 ack。标准偏差条表明典型的消息开销保持较低，例如在 $N=28$ 个成员中，每个协议周期发送消息的开销小于 5 条消息，概率为 0.99。

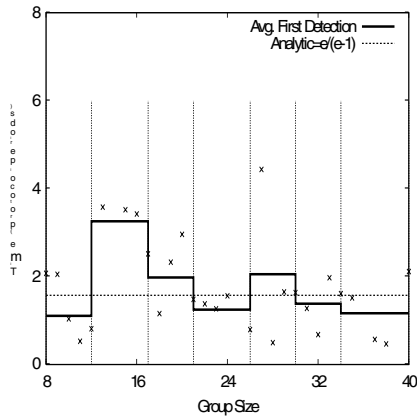
回想一下第 4 节中的描述，该图显示了 *SWIM+Inf* 中每个成员的 *total* 消息开销。和 *SWIM+Inf.+Susp.*，而 *SWIM:Basic* 中只有基本负载（其中附加多播传播成员资格更新）。

5.2. 会员更新的检测和传播延迟

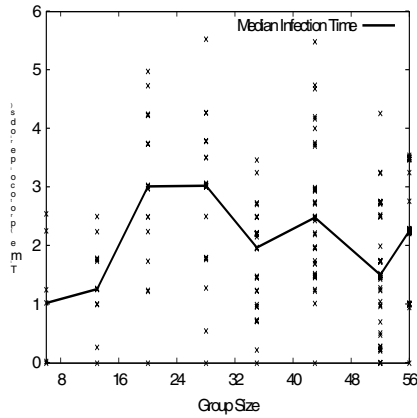
图 3 显示了三种不同协议的故障检测和传播时间的分布情况。图 3(a) 显示了进程故障与在某个非故障组成员处首次检测到进程故障之间的平均时间。平均检测时间似乎与群体大小无关，因此与分析估计相匹配。在 *SWIM:Basic* 协议中，纵轴值对应于过程故障与其故障通知多播时刻之间的间隔。在 *SWIM+Inf.*和游泳+信息。+暂停。协议中，这些值分别对应于故障与故障通知和可疑通知感染开始传播之间的时间间隔。

图 3(b) 绘制了感染式成员更新传播延迟随组大小的变化。传播的中位潜伏期始终只有几个协议周期，并且似乎随着群体规模的大小而缓慢上升（回想一下第 4.1 节的分析预测平均感染时间随群体规模的对数变化）。

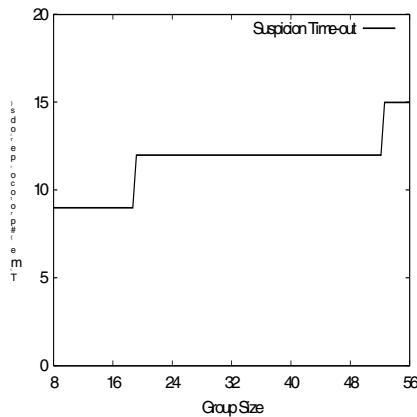
This parameter could be adapted to high membership update rates, thus trading off overhead for dissemination time.



(a) 对于三个协议，首次检测到过程故障的时间（点）随组大小的变化，以及在所示间隔（浅垂直线）中获得的平均值（深色水平线）。还显示了第 3.1 节的分析估计。



(b) SWIM+Inf 中感染通过群体传播的潜伏期。和游泳+信息。+暂停。显示的点对应于不同组成员首次感染的时间。



(c) Suspicion time-out used in the SWIM+Inf.+Susp. protocol.

图 3. 成员资格更新的检测和传播时间。

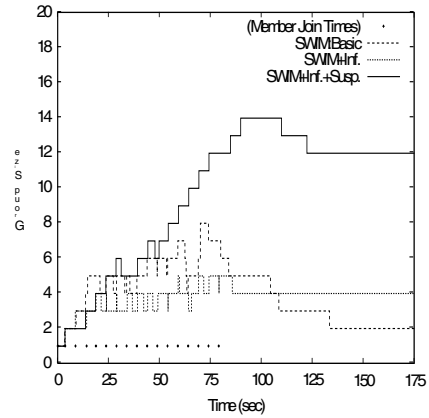


图 4. 使用保守的协议参数时，10% 的数据包丢失率对成员加入序列（底部的点）的影响。

图 3(c) 显示了怀疑超时 ($3l(N+1)$)，并且仅适用于 SWIM+Inf.+Susp. 协议。

从进程失败到从其他成员的成员列表中删除之间的平均时间是从图 3 中的三个图中获得的。对于 SWIM: Basic 协议，这是图 3 (a) 中绘制的值加上网络多播所花费的时间。对于 SWIM+Inf. 对于 SWIM+Inf.+Susp. 协议，这是图 3(a) 和图 3(b) 纵轴上的分布之和。协议中，该延迟是图 3 中三个图上的分布之和。

5.3.故障检测误报

SWIM+Inf.+Susp 的运行。图 3 中的协议生成的错误故障检测数量微不足道。为了评估感染和怀疑机制的相对效益，我们合成了 10% 的高人工丢包率。每个数据包（包括 SWIM:Basic 中的每个网络多播）均以该丢失概率在接收器处丢弃。在我们的实验中，有 17 个进程尝试依次加入 SWIM 组，如图 4 所示。该图显示了加入阶段期间和之后组大小的变化，尽管丢包率持续存在。在最后一次观察到的成员资格变化之后，该图在 175 秒处中断。该图显示了怀疑机制的好处：而 SWIM+Inf.+Susp. - 达到 12 名成员的稳定团体规模、SWIM: Basic 和 SWIM+Inf. 分别稳定在 2 人和 4 人。

图 4 中的数据仅适用于所描述的实验设置。更积极地设置参数 K 和怀疑超时将提高三个协议的稳健性。也改变怀疑超时

提供了一个旋钮来权衡故障检测时间和误报频率。我们将这些问题的调查留到以后的文章中进行。

6. 结论和未来工作

我们提出了 SWIM 的设计、实施和性能评估，这是一种可扩展的弱一致过程组成员协议。SWIM 项目的动机是基于心跳的协议的不可扩展性，这种协议在当今的分布式系统设计者中很流行。

SWIM 的解决方案基于故障检测器和问题的成员更新传播组件的分离。SWIM 故障检测器通过避免心跳并使用随机的进程点对点探测来实现可扩展性。这为组成员提供了恒定的开销，以及恒定的预期故障检测时间。通过搭载故障检测器协议生成的数据包，成员资格更新可以以感染方式（流行方式）高效可靠地传播。添加怀疑机制（具有虚拟化身编号）可减少误报频率，同时权衡故障检测时间。该协议的最终扩展保证了对每个无故障进程的故障的时间限制检测。

通过权衡 ping 目标选择与拓扑信息，SWIM 可以扩展到广域网（WAN）或虚拟专用网（VPN），从而减少网络内部核心网络元素的带宽使用[16]。我们目前正在评估此功能。

本文描述了特定背景下 SWIM 的设计和实施，但我们的结果更具有普遍适用性。尽管 SWIM 的设计针对大型流程组，但我们的分析和实验结果表明，如果在中等规模的子组中使用，这种替代全对所有分布式心跳的方法可以使开销减少一个数量级。分布式哈希表（DHT）系统（例如 Chord、Pastry、Opus [15]）中的副本组。怀疑机制通常适用于任何具有独特故障检测和会员更新组件设计的会员系统。

当今互联网中大规模分布式应用程序的持续扩散程度将取决于这些系统中运行的协议的可扩展且高效的设计。SWIM 为这些应用程序的组成员资格组件提供了这样的解决方案。

致谢：我们感谢 Werner Vogels 在访问和使用 Galaxy 集群方面提供的帮助。我们还感谢 Ben Atkin、Ken Birman、Alan Demers、Robbert van Renesse 和匿名审稿人对草稿的审阅。

参考

- [1] T. Anker、D. Breitgand、D. Dolev 和 Z. Levy。CONGRESS：面向连接的组地址解析服务。*Proc. SPIE'97 on Broadband Networking Technologies*, 1997 年。[2] N. Bailey。*The Mathematical Theory of Infectious Diseases and its Applications*。哈夫纳出版社，第二版，1975 年。[3] K. P. Birman。可靠的分布式计算的进程组方法。*Comm. of the ACM*, 36(12):37–53, 1993。[4] K. P. Birman、M. Hayden、O. Ozkasap、Z. Xiao、M. Budiu 和 Y. Minsky。双模多播。*ACM Trans. on Computer Systems*, 17(2):41–88, 1999。[5] C.E.Perkins 和 E.Royer。即席按需距离矢量路由。*Proc. 2nd IEEE Workshop on Mobile Computing Systems and Applications*, 第 90-100 页，1999 年。[6] T. D. Chandra 和 S. Toueg。用于可靠分布式系统的不可靠故障检测器。*Journ. of the ACM*, 43(2):225–267, 1996。[7] W. Chen、S. Toueg 和 M. K. Aguilera。关于故障检测器的服务质量。*Proc. 30th Intl. Conf. on Dependable Systems and Networks*, 第 191–200 页，2000 年。[8] A. Demers、D. Greene、C. Hauser、W. Irish 和 J. Larson。用于复制数据库维护的流行算法。在 *Proc. 6th Annual ACM Symp. Principles of Distributed Computing*, 第 1-12 页。ACM 出版社，1987 年。[9] S. A. Fakhouri、G. Goldszmidt、I. Gupta、M. Kalantar 和 J. A. Pershing。Gulfstream - 多域服务器群中的动态拓扑管理系统。见 *Proc. 3rd IEEE Intl. Conf. on Cluster Computing*, 第 55-62 页，2001 年。[10] M. J. Fischer、N. A. Lynch 和 M. S. Paterson。不可能通过一个错误的流程达成分布式共识。*Journ. of the ACM*, 32(2):374–382, 1985。[11] I. Gupta、K. Birman 和 R. van Renesse。以毒攻毒：使用随机八卦来对抗随机可扩展性限制。出现在 *The Journ. of Quality and Reliability Engineering International*, 2002 年。[12] I. Gupta、T. D. Chandra 和 G. S. Goldszmidt。关于可扩展且高效的分布式故障检测器。在 *Proc. 20th Annual ACM Symp. on Principles of Distributed Computing*, 第 170-179 页。ACM 出版社，2001 年。[13] A.-M. Kermarrec、L. Massoulié 和 A. Ganesh。大规模信息传播系统中的可靠概率通信。技术报告 MMSR-TR-2000-105，微软研究院，英国剑桥，2000 年。[14] K. Petersen、M. Spreitzer、D. Terry、M. Theimer 和 A.J.Demers。弱一致性复制的灵活更新传播。在 *Proc. 16th ACM Symp. on Operating Systems Principles*, 第 3-6 页。ACM 出版社，1997 年。[15] A. Rowstron 和 F. Kaashoek，编辑。*Proc. 1st Intl. Workshop on Peer-to-Peer Systems*。Springer-Verlag，2002。[16] R. van Renesse、Y. Minsky 和 M. Hayden。八卦式故障检测服务。*Proc. Middleware*, 第 55-70 页，1998 年。

<http://www.cs.cornell.edu/gupta/swim>